Application Tracing



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



strace



strace

System call tracer - https://strace.io

- Available on all GNU/Linux systems Can be built by your cross-compiling toolchain generator or by your build system.
- Allows to see what any of your processes is doing: accessing files, allocating memory... Often sufficient to find simple bugs.
- Usage:
  `strace <command>` (starting a new process)
  `strace -f <command>` (**f**ollow child processes too)
  `strace -p <pid>` (tracing an existing process)
  `strace -c <command>` (time statistics per system call)
  `strace -e <expr> <command>` (use **e**xpression for advanced filtering)

See the strace manual for details



Image credits: https://strace.io/



strace example output

```
> strace cat Makefile
[...]
fstat64(3, {st_mode=S_IFREG|0644, st_size=111585, ...}) = 0
mmap2(NULL, 111585, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb7f69000
close(3) = 0
access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT (No such file or directory)
open("/lib/tls/i686/cmov/libc.so.6", O_RDONLY) = 3
read(3, "\177ELF\1\1\1\0\0\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0320h\1\0004\0\0\0344"..., 512) = 512
fstat64(3, {st_mode=S_IFREG|0755, st_size=1442180, ...}) = 0
mmap2(NULL, 1451632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xb7e06000
mprotect(0xb7f62000, 4096, PROT_NONE) = 0
mmap2(0xb7f66000, 9840, PROT_READ|PROT_WRITE,
    MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xb7f66000
close(3) = 0
[...]
openat(AT_FDCWD, "Makefile", O_RDONLY)  = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=173, ...}, AT_EMPTY_PATH) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7290d28000
read(3, "ifneq ($(KERNELRELEASE),)\nobj-m "..., 131072) = 173
write(1, "ifneq ($(KERNELRELEASE),)\nobj-m "..., 173ifneq ($(KERNELRELEASE),)
```

Hint: follow the open file descriptors returned by `open()`. This tells you what files are handled by further system calls.



strace filtering

- Display only a specific set of system calls:

```
$ strace -e 'openat,write' cat Makefile
```

- Filter out specific system calls:

```
$ strace -e '!poll' cat Makefile
```

- Show only system calls returning a specific status

```
$ strace -e 'status=failed' cat Makefile
```

- Trace how a file is accessed and used among different system calls

```
$ strace -P '/etc/ld.so.cache' cat Makefile
```

- Run `strace --tips` to learn new commands !



ltrace



ltrace

A tool to trace **shared** library calls used by a program and all the signals it receives

- Very useful complement to `strace`, which shows only system calls.
- Of course, works even if you don't have the sources
- Allows to filter library calls with regular expressions, or just by a list of function names.
- With the `-S` option it shows system calls too!
- Also offers a summary with its `-c` option.
- Manual page: https://linux.die.net/man/1/ltrace
- Works better with *glibc*. `ltrace` used to be broken with *uClibc* (now fixed), and is not supported with *Musl* (Buildroot 2022.11 status).

See https://en.wikipedia.org/wiki/Ltrace for details



ltrace example output

```
# ltrace ffmpeg -f video4linux2 -video_size 544x288 -input_format mjpeg -i /dev/video0 \
     -pix_fmt rgb565le -f fbdev /dev/fb0
__libc_start_main([ "ffmpeg", "-f", "video4linux2", "-video_size"... ] <unfinished ...>
setvbuf(0xb6a0ec80, nil, 2, 0)                    = 0
av_log_set_flags(1, 0, 1, 0)                      = 1
strchr("f", ':')                                  = nil strlen("f")                                  =
1
strncmp("f", "L", 1)                              = 26
strncmp("f", "h", 1)                              = -2
strncmp("f", "?", 1)                              = 39
strncmp("f", "help", 1)                           = -2
strncmp("f", "-help", 1)                          = 57
strncmp("f", "version", 1)                        = -16
strncmp("f", "buildconf", 1)                      = 4
strncmp("f", "formats", 1)                        = 0
strlen("formats")                                 = 7
strncmp("f", "muxers", 1)                         = -7
strncmp("f", "demuxers", 1)                       = 2
strncmp("f", "devices", 1)                        = 2
strncmp("f", "codecs", 1)                         = 3
...
```



ltrace summary

Example summary at the end of the ltrace output (`-c` option)

```
% time     seconds  usecs/call     calls      function
------ ----------- ----------- --------- --------------------
 52.64    5.958660     5958660         1 __libc_start_main
 20.64    2.336331     2336331         1 avformat_find_stream_info
 14.87    1.682895         421      3995 strncmp
  7.17    0.811210      811210         1 avformat_open_input
  0.75    0.085290         584       146 av_freep
  0.49    0.055150         434       127 strlen
  0.29    0.033008         660        50 av_log
  0.22    0.025090         464        54 strcmp
  0.20    0.022836       22836         1 avformat_close_input
  0.16    0.017788         635        28 av_dict_free
  0.15    0.016819         646        26 av_dict_get
  0.15    0.016753         440        38 strchr
  0.13    0.014536         581        25 memset
...
------ ----------- ----------- --------- --------------------
100.00   11.318773                4762 total
```



LD_PRELOAD



Shared libraries

- Shared libraries are provided as *.so* files that are actually ELF files
  - Loaded at startup by `ld.so` (the dynamic loader)
  - Or at runtime using `dlopen()` from your code
- When starting a program (an ELF file actually), the kernel will parse it and load the interpreter that needs to be invoked.
  - Most of the time `PT_INTERP` program header of the ELF file is set to `ld-linux.so`.
- At loading time, the dynamic loader `ld.so` will resolve all the symbols that are present in dynamic libraries.
- Shared libraries are loaded only once by the OS and then mappings are created for each application that uses the library.
  - This allows to reduce the memory used by libraries.



Hooking Library Calls

- In order to do some more complex library call hooks, one can use the *LD_PRELOAD* environment variable.
- *LD_PRELOAD* is used to specify a shared library that will be loaded before any other library by the dynamic loader.
- Allows to intercept all library calls by preloading another library.
  - Overrides libraries symbols that have the same name.
  - Allows to redefine only a few specific symbols.
  - "Real" symbol can still be loaded and used with `dlsym` (man 3 dlsym)
- Used by some debugging/tracing libraries (*libsegfault*, *libefence*)
- Works for C and C++.



LD_PRELOAD example 1/2

- Library snippet that we want to preload using *LD_PRELOAD*:

```
#include <string.h>
#include <unistd.h>

ssize_t read(int fd, void *data, size_t size) {
  memset(data, 0x42, size);
  return size;
}
```

- Compilation of the library for *LD_PRELOAD* usage:

```
$ gcc -shared -fPIC -o my_lib.so my_lib.c
```

- Preloading the new library using *LD_PRELOAD*:

```
$ LD_PRELOAD=./my_lib.so ./exe
```



LD_PRELOAD example 2/2

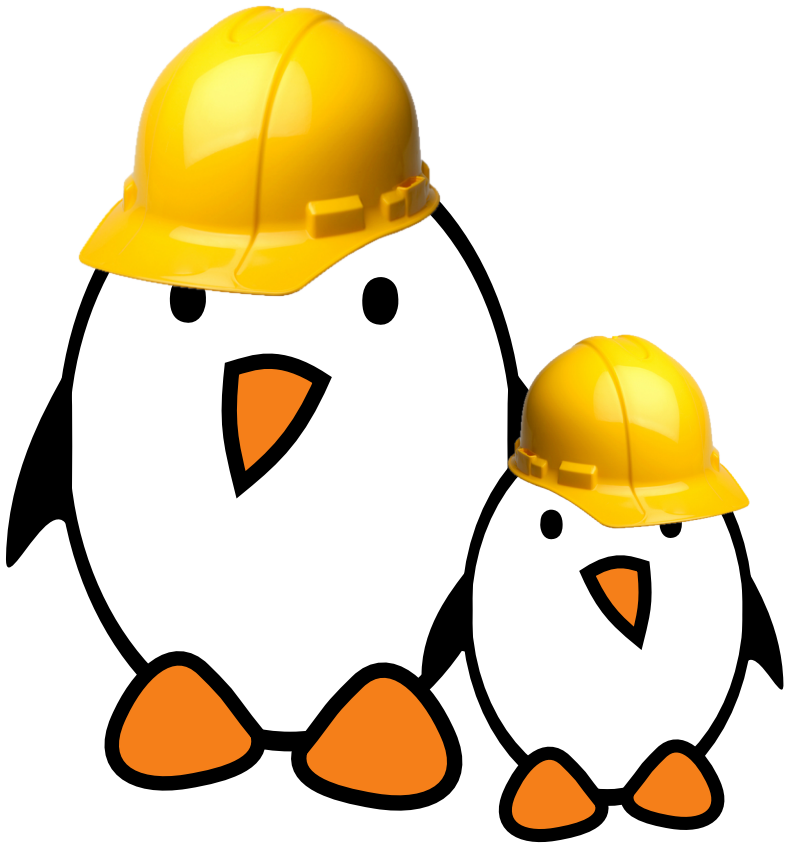
- Chaining a call to the real symbol to avoid altering the application behavior:

```c
#include <stdio.h>
#include <unistd.h>
#include <dlfcn.h>

ssize_t read(int fd, void *data, size_t size)
{
    size_t (*read_func)(int, void *, size_t);
    char *error;

    read_func = dlsym(RTLD_NEXT, "read");
    if (!read_func) {
        fprintf(stderr, "Can not find read symbol: %s\n", dlerror());
        return 0;
    }
    fprintf(stderr, "Trying to read %lu bytes to %p from file descriptor %d\n", size, data, fd);
    return read_func(fd, data, size);
}
```

Practical lab - Application tracing



Analyzing of application interactions

- Analyze dynamic library calls from an application using *ltrace*.
- Overriding a library function with `LD_PRELOAD`.
- Using *strace* to analyze program syscalls.



uprobes and perf



Probes in linux

- The linux kernel is able to dynamically add some instrumentation (or "**probes**") to almost any code running on a platform, either in userspace, kernel space, or both.
- This mechanism works by "patching" the code at runtime to insert the probe. When the patched code is executed, the probe records the execution. It can also collect additional data.
- There are different kinds of probes exposed by the kernel:
  - **uprobes**: hook on almost any userspace instruction and capture local data
  - **uretprobes**: hook on userspace function exit and capture return value
  - **entry fprobe**: hook on kernel function entry
  - **exit fprobe**: hook on kernel function exit
  - **kprobes**: hook on almost any kernel instruction and capture local data
  - **kretprobe**: hook on kernel function exit and capture return value



uprobes

- *uprobe* is a probe mechanism offered by the kernel allowing to trace userspace code.
- Can target any userspace instruction
  - Internally patches the loaded `.text` section with breakpoints that are handled by the kernel trace system
- Exposed by file `/sys/kernel/tracing/uprobe_events`
- User is expected to compute the offset of the targeted instruction inside the corresponding VMA (containing the `.text` section) of the targeted process

```
echo 'p /bin/bash:0x4245c0' > /sys/kernel/tracing/uprobe_events
```

- Uprobes are wrapped by some common tools (e.g: `perf`, `bcc`) for easier usage
- trace/uprobetracer



The perf tool

- *perf* tool was started as a tool to profile application under Linux using performance counters (man 1 perf).
- It became much more than that and now allows to manage tracepoints, kprobes and uprobes.
- *perf* can profile both user-space and kernel-space execution.
- *perf* is based on the `perf_event` interface that is exposed by the kernel.
  - Needs CONFIG_PERF_EVENTS=y at kernel build time
- Provides a set of operations, each having specific arguments (see *perf* help).
  - `stat`, `record`, `report`, `top`, `annotate`, `ftrace`, `list`, `probe`, etc
- Some of those commands operate on an intermediate `perf.data`, containing data from a recording session.



Probing userspace functions

- List functions that can be probed in a specific executable:

```
$ perf probe --source=<source_dir> -x my_app -F
```

- List lines number that can be probed in a specific executable/function:

```
$ perf probe --source=<source_dir> -x my_app -L my_func
```

- Create uprobes on user-space library/executable functions:

```
$ perf probe -x /lib/libc.so.6 printf
$ perf probe -x my_app my_func:3 my_var
$ perf probe -x my_app my_func%return \$retval
```

- Record the execution of these tracepoints:

```
$ perf record -e probe_my_app:my_func_L3 -e probe_libc:printf
```